

Type-Driven Tokenization for Brahmic Scripts

A Pearl

Sai Hemanth Kapila
Microsoft
India
skapila@microsoft.com

Rakshika Bagavathy
Microsoft
India
rakb@microsoft.com

Abstract

Standard tokenizers used in large language models produce malformed text when applied to Brahmic scripts. They are a family of abugidas, writing systems whose consonants carry an inherent vowel that dependent marks can modify. They include Devanagari, Telugu, Tamil, Kannada, and others. The underlying issue is that these tokenizers violate orthographic constraints that do not arise in alphabetic scripts like English. We observe that while English orthography forms a *semigroup* (any two valid tokens can be freely concatenated), Brahmic orthography forms a *partial semigroup*: not every concatenation yields a valid string. We formalise this distinction in Agda, model valid Brahmic tokens as chains in a transition system, and derive a provably correct `fixToken` function that extends any candidate token to respect orthographic boundaries. We then show how this formal derivation translates into a practical patch for SentencePiece as well as a standalone Rust-based pre-tokenizer library, eliminating the observed errors across Indic scripts.

CCS Concepts: • **Theory of computation** → **Type theory**; • **Software and its engineering** → *Functional languages*; • **Computing methodologies** → *Natural language processing*.

Keywords: tokenization, Brahmic scripts, dependent types, Agda, partial semigroup, orthography

ACM Reference Format:

Sai Hemanth Kapila and Rakshika Bagavathy. 2026. Type-Driven Tokenization for Brahmic Scripts: A Pearl. In *Proceedings of The ACM SIGPLAN International Workshop on Type-Driven Development (TyDe '26)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

TyDe '26, Paris, France

© 2026 Copyright held by the owner/author(s).

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Digital text is encoded as a sequence of Unicode *code points*, but a code point need not correspond to a complete written character. A single grapheme, or orthographic unit, may comprise a base character and one or more combining marks, while a word may comprise several such units. Text-processing systems rarely operate directly on these raw code points. Search engines, spell checkers, machine-translation systems, and, most prominently today, large language models (LLMs) rely on a process called *tokenization*, which segments text into reusable units called *tokens*. In this paper, tokens are *subwords*, vocabulary entries that may contain several code points but may be shorter than a word. A token is therefore atomic to the model, even though it may have internal Unicode and orthographic structure.

Modern systems learn their inventory of tokens, their *vocabulary*, from corpus statistics. The two dominant algorithms, Byte Pair Encoding (BPE) [16] and Unigram [9], used in GPT- and Llama-style architectures, build vocabularies of frequently co-occurring code-point sequences and then split any input into a sequence of vocabulary entries. Such *frequency-driven subword tokenizers* know nothing about the writing system they operate on. Any sufficiently frequent sequence of code points can become a token, and a token boundary may fall between any two code points.

For English this indifference is harmless, because every letter is a self-contained symbol. In other words, any fragment of valid text is itself displayable text. However, the same cannot be said about the Brahmic scripts of South and South-East Asia. These scripts are *abugidas* [2], where each consonant letter carries an inherent vowel (Telugu గ reads *ka*), and other vowels are written as *dependent* marks attached to a consonant rather than as free-standing letters. A script's *orthography* determines which character sequences constitute valid written text and therefore constrains which code points may occur in succession. Standard “out-of-the-box” tokenizers routinely violate these constraints. Our experiments training LLMs on Telugu, Tamil, Devanagari, and Gujarati corpora show frequent overlapping characters, unreadable sequences, and isolated dependent modifiers rendered as empty dotted circles (e.g., ీ).

Recent empirical studies confirm that this category of errors is widespread across complex writing systems. Manodnya and Giri [13] showed that treating *orthographic syllables* as the atomic unit for Indic language modelling yields a 30% improvement in compression ratio over BPE and significantly better language-model performance. This is primarily because BPE produces tokens that are not valid character sequences in abugida scripts. De Nardi and Manodnya [6] extended this finding to Arabic-script languages in a multi-lingual evaluation.

The problem is therefore well-established. Frequency-driven subword tokenizers violate orthographic constraints that do not arise in languages like English. *What has been missing is a formal account of why these violations occur and a provably correct fix.*

In this pearl, we supply that account. We observe that the distinction is *algebraic*. English orthography, viewed as an operation on character sequences, forms a *semigroup*, where concatenation of any two valid tokens always yields a valid token. Brahmic scripts (Devanagari, Telugu, Tamil, Kannada, and others) contain *dependent characters* like vowel signs and vowel suffixes, that are meaningful only when attached to a preceding base character. Other orthographies, such as Thai or Vietnamese in decomposed Unicode form, show similar properties. These orthographies form a *partial semigroup*, where concatenation may fail when the boundary between two sequences violates a transition constraint. Tokenizers designed for semigroups produce invalid output when applied to partial semigroups. Although we quantify the damage with language-modelling metrics, the constraint itself is not an artifact of LLMs. Rather it is a fact about the computational representation of writing systems. It applies to any system that splits Unicode text, including a renderer, search index, or tokenizer.

We formalise this distinction in Agda and derive a provably correct token-boundary fix:

1. We model orthographic structure at two algebraic levels: total and partial semigroups, in Agda (Section 2).
2. We encode Brahmic orthography as a typed transition system. We then define valid tokens as chains in this system, and derive a `fixToken` function with machine-checked proofs of validity preservation, remainder correctness, and completeness (Section 3).
3. We prove that Brahmic script is a partial semigroup, and to illustrate that the pattern applies beyond this family, we provide a proof for Vietnamese too. (Section 3).
4. We translate the formal derivation into a patch for SentencePiece, Kudo and Richardson’s open-source library for training and applying BPE and Unigram tokenizers [10]. We also provide a configurable Rust pre-tokenizer library, eliminating observed errors in Telugu, Tamil, Hindi, and Gujarati (Section 4). At the

cost of a modest (about 10%) increase in token count, the resulting tokenization cuts language-model perplexity nearly in half (from 166 to 85 for Unigram, and from 159 to 92 for BPE) on a Telugu corpus (Section 5).

2 Orthography as Algebraic Structure

This section first reviews the character classes that make up Brahmic scripts, and shows with a concrete Telugu example, how a badly placed token boundary produces malformed text. It then captures the difference between alphabetic and Brahmic orthographies as two algebraic structures in Agda, a semigroup and a partial semigroup.

2.1 Brahmic Script Characteristics

Brahmic scripts are abugidas where each consonant letter inherently carries a vowel (typically ‘a’). The character classes relevant to tokenization are:

- **Consonant symbols:** carry an inherent vowel (e.g., Telugu *ka* = U+0C15).
- **Independent vowels:** stand-alone vowel letters (e.g., Telugu *a* = U+0C05).
- **Dependent vowels (matras):** modify a consonant’s inherent vowel. These cannot appear independently (e.g., Telugu *-i* = U+0C3F).
- **Virama (halant):** cancels the inherent vowel, creating a *dead* consonant that joins with the next (e.g., U+0C4D).
- **Vowel suffixes:** nasalisation or aspiration markers that follow a vowel (e.g., Telugu anusvara = U+0C02).

A concrete example. Consider the Telugu word *vijña* (a prefix meaning “knowledge”), encoded as the Unicode sequence in Table 1.

Codepoint	Glyph	Role	Can start?
U+0C35	క	Consonant	Yes
U+0C3F	ి	Dependent vowel	No
U+0C1C	జ	Consonant	Yes
U+0C4D	‌	Virama	No
U+0C1E	ఖ	Consonant	Yes
U+0C3E	ి	Dependent vowel	No

Table 1. Decomposition of the Telugu syllable cluster విజ్ఞా (*vijñā*). Characters marked **No** are dependent, they cannot begin a valid token.

If a tokenizer splits this sequence after position 3 (U+0C1C, the consonant *ja*), the right fragment begins with the virama U+0C4D. If an LLM later emits such a token after anything other than a consonant, the result is ill-formed. A rendering engine displays this as a dotted circle (◌) followed by the virama glyph, producing visually corrupted output. Valid tokens must begin with a consonant or independent vowel.

2.2 Two Levels of Orthographic Structure

We model orthography at two algebraic levels in Agda.

Level 1: Semigroup (English, German). Any two valid tokens can always be combined and the combination operation is associative.

```
record Orthography1 (Token : Set)
  : Set where
  field
    combine : Token → Token → Token
    assoc  : ∀ (x y z : Token)
      → combine (combine x y) z
      ≡ combine x (combine y z)
```

Level 2: Partial Semigroup. In scripts such as Brahmic and Vietnamese, combination may fail when tokens are orthographically incompatible.

```
record Orthography2 (Token : Set)
  : Set where
  field
    combine : Token → Token → Maybe Token
    assoc  : ∀ (x y z : Token)
      (xy xyz : Token)
      → combine x y ≡ just xy
      → combine xy z ≡ just xyz
      → ∃ λ yz
        → combine y z ≡ just yz
        × combine x yz ≡ just xyz
```

BPE and Unigram tokenizers implicitly assume the total structure Orthography_1 . For scripts living in Orthography_2 , we need a tokenizer that respects the partial structure.

3 Formal Model of Brahmic Tokenization

We now make the picture of Section 2 precise. We encode the Brahmic character classes and their permitted transitions as Agda datatypes, define valid tokens as chains in the resulting transition system, and derive a boundary-repairing function `fixToken` together with machine-checked proofs of its correctness. We then replay the same construction for Vietnamese and close by exhibiting both scripts as instances of the partial-semigroup record.

3.1 Character Types and Transitions

We model Brahmic character categories as a finite type and valid transitions as a relation. Figure 1 shows the permitted transitions between character classes. An edge from A to B means a character of class B may immediately follow one of class A within a valid token.

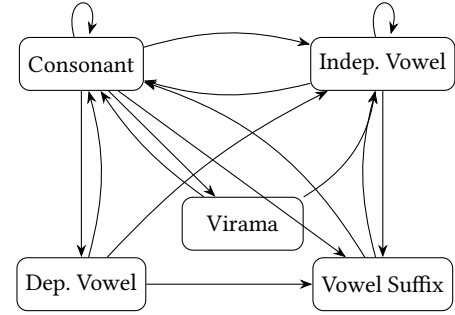


Figure 1. Valid transitions between Brahmic character classes. Edges represent the constructor names in the Agda formalisation (e.g., the edge $\text{Consonant} \rightarrow \text{Virama}$ corresponds to `DeadConsonant`).

```
data UnicodeBrahmic : Set where
  Consonant      : UnicodeBrahmic
  IndependentVowel : UnicodeBrahmic
  DependentVowel : UnicodeBrahmic
  VowelSuffix    : UnicodeBrahmic
  Virama         : UnicodeBrahmic

data _→_ : UnicodeBrahmic
  → UnicodeBrahmic → Set where
  DeadConsonant : Consonant → Virama
  VowelSymbol   : Consonant → DependentVowel
  NextVowel     : ∀ {t} → t → IndependentVowel
  NextConsonant : ∀ {t} → t → Consonant
  VowelSuffix1 :
    Consonant → VowelSuffix
  VowelSuffix2 :
    IndependentVowel → VowelSuffix
  VowelSuffix3 :
    DependentVowel → VowelSuffix
```

The transition relation $\rightarrow$ encodes the state machine of valid character sequences. A valid token is a chain of transitions starting from a consonant or independent vowel.

3.2 Valid Chains and Tokens

A `ValidChain` witnesses that every adjacent pair in a character sequence satisfies the transition relation. It is indexed by the *last* element of the chain, which makes appending two chains type-safe, since we can verify that the last element of the first chain can transition to the first element of the second.

```
data ValidChain : UnicodeBrahmic
  → List UnicodeBrahmic → Set where
  base : ∀ {t} → ValidChain t (t :: [])
  step : ∀ {t1 t2 last}
    {rest : List UnicodeBrahmic}
    → (t1 → t2)
    → ValidChain last (t2 :: rest)
    → ValidChain last (t1 :: t2 :: rest)
```

The base constructor witnesses a single-character chain. The step prepends a character by supplying a transition proof. A `ValidBrahmicToken` is then a chain that starts at a permitted initial state.

```
data ValidBrahmicToken
  : List UnicodeBrahmic → Set where
  consonant-start : ∀{last} {rest}
    → ValidChain last (Consonant ::rest)
    → ValidBrahmicToken (Consonant ::rest)
  vowel-start : ∀{last} {rest}
    → ValidChain last
      (IndependentVowel ::rest)
    → ValidBrahmicToken
      (IndependentVowel ::rest)
```

Crucially, there is no constructor for chains beginning with a dependent vowel, virama, or vowel suffix. These are ruled out *by construction*. A dependent character can never begin a valid token.

A key closure property follows: any two valid tokens can be concatenated to form a valid token (via `NextConsonant` or `NextVowel` at the boundary). This asserts that the set of valid tokens forms a semigroup under concatenation. This closure property does not contradict our partial-semigroup characterization. Concatenation is total on the restricted set of already-valid tokens, because the second token necessarily begins with a consonant or independent vowel. Partiality arises on the larger domain of candidate fragments considered by a tokenizer, a fragment may begin with a dependent character, and concatenating it is defined only when the boundary transition is permitted. Note that valid tokens are inherently non-empty (both constructors require at least one character). There is no unit element, which is why the appropriate algebraic structure is a semigroup rather than a monoid.

3.3 The `fixToken` Function

To see where types must meet practice, recall that a subword tokenizer processes text from left to right, selecting a vocabulary entry that matches a prefix of the remaining input, emitting it, and continuing with the rest of the text. BPE determines this selection through greedy merging, whereas Unigram uses a likelihood criterion. The right boundary of every emitted token is thus chosen by frequency, with no regard for the script. The specification developed above can be stated in one sentence: *never place a token boundary where the fragment to its right would begin with a dependent character*.

At the heart of our approach is a recursive function, `fixToken`, that repairs a boundary proposed by a standard tokenizer so that it meets this specification. It is designed to wrap an existing BPE or Unigram tokenizer as a post-processing step. Given a candidate token (already identified by BPE or Unigram) and the remaining input, the function performs a one-character lookahead. If the next character

is a valid token start (a consonant or independent vowel), the candidate is complete and returned as is. Otherwise, the next character is a dependent continuation that *must* be absorbed into the current token, so append it and recur.

```
fixToken : List UnicodeBrahmic
  → List UnicodeBrahmic
  → List UnicodeBrahmic
  × List UnicodeBrahmic
fixToken cand [] = cand , []
fixToken cand (Consonant ::r)
  = cand , Consonant ::r
fixToken cand (IndependentVowel ::r)
  = cand , IndependentVowel ::r
fixToken cand (DependentVowel ::r)
  = fixToken
    (cand ++ DependentVowel ::[]) r
fixToken cand (VowelSuffix ::r)
  = fixToken (cand ++ VowelSuffix ::[]) r
fixToken cand (Virama ::r)
  = fixToken (cand ++ Virama ::[]) r
```

Repairing the Telugu example. Consider again the split from Table 1. The tokenizer has proposed the candidate U+0C35 U+0C3F U+0C1C, whose character classes are Consonant, DependentVowel, and Consonant. The remainder is U+0C4D U+0C1E U+0C3E, classified as Virama, Consonant, and DependentVowel. On its first step, `fixToken` observes that the remainder begins with a virama, which cannot start a valid token. It therefore appends U+0C4D to the candidate and recurses on the remaining U+0C1E U+0C3E. The second step sees that U+0C1E is a consonant and hence a valid token start, so the recursion stops. The function returns U+0C35 U+0C3F U+0C1C U+0C4D as the repaired token and U+0C1E U+0C3E as the remainder. Thus the boundary moves one code point to the right, no character is lost, and the remainder now begins at a valid boundary.

The function terminates because the remainder shrinks by one element at each recursive call. It returns a pair: the extended token and the unconsumed remainder. The lookahead is the pattern match on the head of the second argument. There is no need for backtracking or unbounded search.

3.4 Correctness Properties

We prove three key properties that together guarantee that `fixToken` is a safe, lossless post-processing step:

Validity preservation. The lemma establishes that token validity is preserved on well-formed input. If the candidate is a valid token and the full concatenation (candidate appended with remainder) is also valid, then the extended token returned by `fixToken` is valid:

```
fixTokenValid : ∀{l1 l2}
  →ValidBrahmicToken l1
  →ValidBrahmicToken (l1 ++ l2)
  →let (parsed , _) = fixToken l1 l2
  in ValidBrahmicToken parsed
```

Remainder validity. The remainder always begins with a valid start character (or is empty), ensuring that the *next* token can also be validly formed. The predicate `RemainderOk` states exactly this, reusing the `IsValidBrahmicStart` type that singles out the two permitted initial states:

```
data IsValidBrahmicStart
  : UnicodeBrahmic →Set where
  ConsonantStart :
    IsValidBrahmicStart Consonant
  IndependentVowelStart :
    IsValidBrahmicStart IndependentVowel

RemainderOk
  : List UnicodeBrahmic →Set
RemainderOk [] = ⊤
RemainderOk (x ::_) =
  IsValidBrahmicStart x

fixTokenRemainderOk : ∀(l1 l2)
  →let ( _ , rem ) = fixToken l1 l2
  in RemainderOk rem
```

Completeness. This lemma asserts that no characters are lost or duplicated. The extended token concatenated with the remainder equals the original input:

```
fixTokenComplete : ∀(l1 l2)
  →let (parsed , rem) = fixToken l1 l2
  in parsed ++ rem ≡ l1 ++ l2
```

Together, these three properties ensure that `fixToken`, applied as a post-processing step after any standard tokenizer, produces a valid tokenization. Every token respects orthographic constraints and the original text is faithfully preserved.

3.5 Vietnamese: A Simpler Instance

The same pattern applies to other partial-semigroup orthographies. Vietnamese in decomposed Unicode form provides a second example. The orthography is simpler than Brahmic (three character classes instead of five). It is also partial.

The orthographic constraint in Vietnamese is that a base character may carry at most one diacritic (circumflex, breve, horn) followed by at most one tone mark (acute, grave, hook, tilde, dot below). Stacking two diacritics, two tones, or placing a diacritic after a tone are all invalid.

```
data Vietnamese : Set where
  BaseCharacterv : Vietnamese
  Diacriticv : Vietnamese
  Tonev : Vietnamese

data _→v_ : Vietnamese →Vietnamese →Set where
  Markv :
    BaseCharacterv →vDiacriticv
  ToneAfterBasev :
    BaseCharacterv →vTonev
  ToneAfterDiacriticv :
    Diacriticv →vTonev
  NextBasev :
    ∀ {t} →t →vBaseCharacterv
```

The invalid transitions (`Diacritic` → `Diacritic`, `Tone` → `Tone`, `Tone` → `Diacritic`) have no constructors, and are thus ruled out by the type, as before.

The `fixToken` function follows the identical structure:

```
fixTokenv cand [] = cand , []
fixTokenv cand (BaseCharacterv ::r)
  = cand , (BaseCharacterv ::r)
fixTokenv cand (Diacriticv ::r)
  = fixTokenv (cand ++ Diacriticv ::[]) r
fixTokenv cand (Tonev ::r)
  = fixTokenv (cand ++ Tonev ::[]) r
```

The same correctness properties (validity preservation, remainder correctness, completeness) hold and are proved analogously.

3.6 Instances of Orthography₂

We close the formal development by constructing concrete `Orthography2` instances for both scripts. For each, we define a decidable boundary-checking function `canFollow` that returns true when the transition relation is inhabited, and a `combine` that concatenates two character lists only when the boundary is valid:

```
combineBrahmic : List UnicodeBrahmic
  →List UnicodeBrahmic
  →Maybe (List UnicodeBrahmic)
combineBrahmic (x ::xs) (y ::ys)
  with canFollow (getLast x xs) y
... | true = just (x ::xs ++ y ::ys)
... | false = nothing
```

We prove that `canFollow` is both sound and complete with respect to `_→_`, and that `combineBrahmic` satisfies associativity, yielding:

```
brahmicOrthography2 :
  Orthography2 (List UnicodeBrahmic)

vietnameseOrthography2 :
  Orthography2 (List Vietnamese)
```


This completes the bridge between the abstract algebraic structure (Section 2) and the concrete transition-based model. Brahmic and Vietnamese character sequences are certified instances of the partial semigroup record.

Artifact availability. The artifact accompanying this paper with complete Agda formalisation used in this work, is available at https://github.com/rakshikab/brahmi_script_formalization/blob/main/formalization/Brahmic.agda. The development type-checks under the `-safe` flag with Agda 2.9.0 and version 2.3 of the Agda standard library.

4 Implementation

The formal model of Section 3 gives us a clear specification. We never allow a token boundary where the right fragment would begin with a dependent character. We now describe two practical realisations of this specification.

4.1 The Dual Problem of Pre-tokenizers

Tokenizer libraries distinguish the tokenizer proper, which learns and applies the subword vocabulary, from a *pre-tokenizer*: a preliminary pass that carves the input into coarse fragments, classically at whitespace and punctuation. The vocabulary is then learned and applied strictly *within* fragments, with no token ever crossing a fragment boundary. SentencePiece [10], the widely used open-source tokenizer library that implements both BPE and Unigram, exposes this mechanism for extension via its `PretokenizerForTrainingInterface`. Users can supply custom logic that forces the tokenizer to *break* at specified positions. This solves the *dual* of our problem. Pre-tokenizers specify where tokens *must* be split. However, we need to specify where tokens *must not* be split. No existing SentencePiece interface supports the latter constraint directly.

4.2 SentencePiece Patch

The most direct translation of our formal result is a filter in SentencePiece’s `IsValidSentencePiece()` function (`src/trainer_interface.cc`), which validates every candidate token during training. We add the following predicate that rejects any candidate whose first character is a dependent or combining mark.

```
static bool IsDependentOrCombining(char32 c) {
  // Telugu dependent vowels + virama
  if (c >= 0x0C3E && c <= 0x0C56) return true;
  if (c == 0x0C4D) return true;
  // Combining diacriticals (Vietnamese)
  if (c >= 0x0300 && c <= 0x036F) return true;
  return false;
}
```

This is the `canFollow` check from our Agda development, specialised to the “is this a valid token start?” question. By rejecting such candidates at training time, the learned vocabulary contains only tokens that begin at valid boundaries,

implying that the `fixToken` lookahead is then never needed at inference time.

Note, however, that this patch hard-codes specific Unicode ranges for Telugu and Vietnamese. Supporting additional Brahmic scripts (Tamil, Kannada, Devanagari, etc.) would require extending the range table for each script. Keeping this maintenance burden in mind, we discuss a more general solution below.

4.3 Rust Pre-tokenizer Library

While the SentencePiece patch works for specific hard-coded Unicode ranges, we wanted a more flexible solution configurable for arbitrary Brahmic scripts without modifying SentencePiece’s source. We built a standalone Rust library that:

1. Pre-tokenizes input text by identifying orthographically valid grapheme clusters using the transition rules (the Rust analogue of `fixToken`).
2. Maps each cluster to an unused Unicode code point from the Private Use Area (PUA), producing a one-to-one encoding where each PUA symbol represents an indivisible orthographic unit.
3. Passes the transformed text to SentencePiece, which now operates on atomic symbols that cannot be split incorrectly.
4. Reverses the mapping when consuming LLM output, recovering valid Brahmic text.

This approach has been tested with Telugu, Tamil, Gujarati, and Devanagari, eliminating orthographic errors across all tested configurations.

The boundary logic at the core of the library is a direct, if manual, transcription of the Agda development: a Rust enum mirrors `UnicodeBrahmic`, and a case analysis on the incoming character’s class, guarded by the class of the preceding symbol, makes exactly the accept/reject decisions of `canFollow`, clause for clause. The code surrounding that core (tables mapping Unicode ranges to character classes for each script, PUA bookkeeping, and the SentencePiece plumbing) is routine engineering with no formal counterpart.

The key insight connecting the implementation back to our formal development is that standard tokenizers cannot subdivide a single Unicode code point. This forces the language model to treat our `ValidBrahmicTokens` as indivisible units. The PUA encoding is the runtime mechanism that enforces what the type system guarantees statically that every token respects orthographic boundaries.

5 Evaluation

A natural concern is whether enforcing orthographic correctness degrades tokenizer efficiency. We evaluate the Rust pre-tokenizer of Section 4 on two metrics, one measuring cost and one measuring benefit. *Normalized Sentence Length*

(NSL) [4] is the ratio of the token count our tokenizer produces for a sentence to the token count of a reference tokenizer (here, GPT-4o's). It captures efficiency, since a model must process and generate more tokens for the same text when NSL rises. *Perplexity* measures downstream language-model quality, how well a model trained on the resulting tokens predicts held-out text, with lower values indicating better predictions.

Setup. We trained SentencePiece Unigram and BPE tokenizers, each with a vocabulary size of 8000, on 75K Telugu Wikipedia articles (12.1M tokens). For each tokenization configuration, we then trained nanoGPT [7], a minimal implementation of GPT-2, on the same corpus and measured perplexity on a held-out set of 1M tokens of Telugu news text [14].

NSL. The Brahmi pre-tokenizer increases NSL marginally, from 0.65 to 0.71 (Unigram) and 0.64 to 0.72 (BPE), remaining well below the GPT-4o baseline of 1.0. The small increase reflects the fact that our approach sometimes prevents merges that cross orthographic boundaries, producing slightly more tokens per sentence. This cost is inherent in the specification, not in the transcription from Agda to Rust: NSL counts tokens rather than running time, and any tokenizer that refuses to split orthographic units must occasionally spend more tokens on the same text, however it is implemented.

Perplexity. The effect on language-model quality is dramatic and positive. Perplexity drops from 166 to **85** (Unigram) and from 159 to **92** (BPE), a reduction of nearly 50%. Because each token now corresponds to a linguistically meaningful unit, the model can learn more coherent representations and predict subsequent tokens far more accurately.

Summary. A marginal 10% increase in token count buys a 50% reduction in perplexity. Orthographic correctness improves downstream model performance. The formal guarantees from Section 3 translate directly into measurable empirical gains.

6 Related Work

Tokenization for non-Latin scripts. Manodnya and Giri [13] introduced Orthographic Syllable Pair Encoding (OSPE), which uses orthographic syllables as the atomic subword unit for Indic languages, achieving 30% better compression than BPE and improved perplexity. De Nardi and Manodnya [6] extended this to Arabic-script languages in a multilingual evaluation, showing that BPE-induced fragmentation causes up to 27-point F1 drops and unstable training. Both works demonstrate the empirical severity of the problem; our contribution is to explain *why* it arises (the semigroup vs. partial semigroup distinction) and to provide a machine-checked formal fix.

Script-aware segmentation before LLMs. Segmenting Indic text at orthographic rather than statistical boundaries predates neural language models. Kunchukuttan and Bhattacharyya [11] used orthographic syllables as the unit of translation for statistical machine translation between related languages, outperforming word-, morpheme-, and character-level units. Mainstream tokenizer libraries also ship script-aware heuristics: SentencePiece can forbid merges that span distinct Unicode scripts, and subword training is usually preceded by regex-based pre-tokenizers keyed on Unicode character categories. These mechanisms prevent some malformed tokens, but they do not model the *within-script* dependency constraints that Brahmic orthography imposes, and they offer no correctness guarantees. Our contribution is accordingly not new boundary rules—script-aware segmenters embody the same linguistic facts—but their formulation as types, which turns “the tokenizer respects the script” from an empirical observation into a machine-checked property.

Types for parsing and for language. Dependently typed and certified parsing is a well-explored area: Brink et al. [3] embed grammars in Agda with types that keep semantic actions consistent with the grammar; Danielsson [5] gives parser combinators that are total by construction; Sarracino et al. [15] verify, in Coq, parsers for regular grammars extended with data dependency. These works certify parsers for programming languages and machine-oriented formats. We apply the same discipline one level lower, to the writing system itself. A separate tradition applies type-theoretic tools to natural language: Lambek’s syntactic calculus types sentence structure [12], Barker and Shan analyse quantification and scope through continuations [1], and Kovalev and Angiuli recently formalised a dependently typed calculus of event structure in Agda [8]. Those works assign types to syntax and semantics. To our knowledge, ours is the first dependently typed account of *orthographic* well-formedness.

Subword tokenization. BPE [16] and Unigram [9] are the dominant subword algorithms, implemented in SentencePiece [10]. Dagan et al. [4] study how to get the most out of tokenizers for pre-training and domain adaptation but do not address orthographic validity. None of these works provide formal guarantees about the well-formedness of learned tokens.

7 Conclusion

Recent work [6, 13] has established that frequency-driven tokenizers systematically fail on Brahmic and Arabic-script languages. We have shown that this failure has an algebraic explanation: these orthographies form partial semigroups, not semigroups, and tokenizers designed for the latter inevitably violate the constraints of the former.

By formalising this distinction in Agda, we derived a provably correct token-fixing function, with machine-checked guarantees of validity preservation, remainder correctness, and completeness. This directly translates into practical tokenizer improvements. The same algebraic pattern applies to Vietnamese and potentially any script with combining or dependent marks. We hope this pearl illustrates that type-driven formalisation can both *explain* an empirically observed failure and deliver a repair whose correctness is machine-checked rather than merely tested.

References

- [1] Chris Barker and Chung-chieh Shan. 2014. *Continuations and Natural Language*. Oxford University Press.
- [2] William Bright. 1999. A Matter of Typology: Alphasyllabaries and Abugidas. *Written Language and Literacy* 2, 1 (1999), 45–55.
- [3] Kasper Brink, Stefan Holdermans, and Andres Löb. 2010. Dependently Typed Grammars. In *Mathematics of Program Construction (MPC 2010) (Lecture Notes in Computer Science, Vol. 6120)*. Springer, 58–79.
- [4] Gautier Dagan, Gabriel Synnaeve, and Baptiste Rozière. 2024. Getting the most out of your tokenizer for pre-training and domain adaptation. arXiv:2402.01035 [cs.CL] <https://arxiv.org/abs/2402.01035>
- [5] Nils Anders Danielsson. 2010. Total Parser Combinators. In *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 285–296.
- [6] Manodnya K H and Luc De Nardi. [n. d.]. Orthographic Structure Matters: Tokenization Failures in Arabic-Script and Related Languages. <https://api.semanticscholar.org/CorpusID:287508942>
- [7] Andrej Karpathy. 2023. nanoGPT. <https://github.com/karpathy/nanoGPT>
- [8] Pavel Kovalev and Carlo Angiuli. 2025. A dependently-typed calculus of event telicity and culminativity. *Mathematical Structures in Computer Science* (2025). arXiv:2506.06968.
- [9] Taku Kudo. 2018. Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates. *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics* (2018), 66–75.
- [10] Taku Kudo and John Richardson. 2018. SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Association for Computational Linguistics, 66–71.
- [11] Anoop Kunchukuttan and Pushpak Bhattacharyya. 2016. Orthographic Syllable as basic unit for SMT between Related Languages. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 1912–1917.
- [12] Joachim Lambek. 1958. The Mathematics of Sentence Structure. *The American Mathematical Monthly* 65, 3 (1958), 154–170.
- [13] KH Manodnya and Animesh Giri. 2023. Orthographic Syllable Pair Encoding for Language Modelling Tasks in Indic Languages. In *2023 IEEE MIT Undergraduate Research Technology Conference (URTC)*. IEEE, 1–6. <https://ieeexplore.ieee.org/document/10534970>
- [14] saidineshpola. 2024. Telugu News Dataset. https://huggingface.co/datasets/saidines12/telugu_news_dataset.
- [15] John Sarracino, Gang Tan, and Greg Morrisett. 2022. Certified Parsing of Dependent Regular Grammars. In *2022 IEEE Security and Privacy Workshops (SPW)*. IEEE, 113–123.
- [16] Rico Sennrich, Barry Haddow, and Alexandra Birch. 2016. Neural Machine Translation of Rare Words with Subword Units. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 1715–1725.